Advanced Machine Learning (AML-MDS)

Course handout

Master in Data Science, Fall 2026–27

Lluís A. Belanche

Version of 14 October 2026

This handout has two parts. **Part I** says how the course works: aims, assessment, calendar, rules. **Part II** is reference material: notation, vocabulary, conventions and pointers. Neither will be read out in class. Keep it beside you for the whole course; administrative matters are on the course website.

This document is alive: check for updates during the course (just look at the date of the version in the footer).

Part I
How this course works

1 Aims

1. To understand what characterises a **learning problem**, and what it means for a machine to learn from data, on a rigorous mathematical basis.
2. To understand the central role of **regularization**, and how to use it in practice.
3. To **extend and unify** algorithms you have already met.
4. To learn some recent methods, particularly **hybrid** ones.
5. To apply all of this to complex **real** data: structured, high-dimensional, mixed-typed data, ...

2 Assessment

Component	Weight	Due
Final examination	25%	15 January 2027, 15:00–18:00
Project proposal	5%	Friday 23 October
Milestone 1 — kernel methods	15%	Friday 20 November
Milestone 2 — neural methods	15%	Friday 18 December
Milestone 3 — hybrid and ensemble methods	15%	with the final report
Oral presentation and defence	10%	mid late January (TBA)
Final integrative report	15%	late January (TBA)

The project is one problem, carried through the whole course. You choose a problem — a dataset and a question about it, or a methodological question studied on synthetic and real data —, it is approved by **28 October**, and every milestone attacks the same problem with the tools of the block just finished. The milestones are short — a PDF of three to five pages, with the code alongside. The writing effort belongs to the final report.
A proposal is a problem, not a choice of method: you do not need to know the methods yet, since every block will be applied to the problem you choose now. What it must contain, what a good dataset looks like and how everything is graded are in two separate documents, the **project guidelines** and the **rubric**. Read them now.

3 Calendar

Theory		**Laboratories** (Fridays)	
14, 18, 21 Sep	Foundations	2 October	0: resampling (on the blackboard)
5, 14, 19, 26 Oct, 2 Nov	Kernel methods	9 October	1: Foundations (extra session)
9, 16, 23 Nov	Neural methods	16 October	2: Kernel methods I
30 Nov	Hybrid methods	30 October	3: Kernel methods II
14, 21 Dec	Ensemble methods	13 November	4: Kernels against networks
		27 November	5: Neural methods
		11 December	6: Hybrid methods

Note that 14 October is a **Monday**, replacing the holiday of 12 October. There is no session on 7 December. The session of 28 September did not take place: its material (F3 and F4) is for self-study. The final integrative report and the oral presentations take place in late January; their exact dates will be announced in the Racó and added here.

4 Software

Either **R** or **Python** is fine for the project; choose one and be fluent in it. The laboratories fix the language per session — R for laboratories 1 to 3, Python and PyTorch for 4 to 6.

5 Two rules

Use of AI assistants. You may use them. If you do, say where, every time — in the code as a comment, and in any text you hand in. The reason is not suspicion: in this course the object of interest is often *why an answer is wrong*, and an unattributed generated answer destroys that. How to use them well, and how to cite them (APA, September 2025), is in Section 14, *Studying with an AI assistant*.

Reproducibility. Set the seed once, at the top. Record package versions. Never edit a result by hand, not even to fix an obvious typo in a number: re-run instead.

6 Where things are

Material, sheets and announcements are posted in the Racó. Four documents will accompany this one: the **laboratory instructions**; the **project guidelines**, with the **rubric** and the **project template**; and the **exercise sheets**, issued one per block.

Part II
Reference

7 Notation

Everything the slides use, in one place. Please report inconsistencies, typos, etc, as soon as you spot them.

Spaces and data	
$\mathcal{X}$, $\mathcal{Y}$	input and output spaces
d, n	number of input variables; number of observations
ρ	the joint distribution on $\mathcal{X} \times \mathcal{Y}$: unknown, fixed, never seen
$\rho_{\mathcal{X}}$	its marginal on the inputs — where the test points come from too, and the reason anything transfers
$D_n = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$	the dataset, drawn i.i.d. from ρ; i indexes observations, j indexes variables
$\boldsymbol{x}$, $\mathbf{X}$	a column vector (bold italic); a matrix (bold upright)
$\mathbf{X}^\top$	transpose; $\mathbf{X}$ is $n \times (d{+}1)$, one observation per row
$\boldsymbol{u} \cdot \boldsymbol{v}$	the inner product in $\mathbb{R}^d$, $\boldsymbol{u}^\top \boldsymbol{v}$; in a space of functions it is written $\langle f, g \rangle_{\mathcal{H}}$
ε, σ^2	the noise *term* added to a target, and its variance — distinct from $\nu(\boldsymbol{x})$ below, which is the noise *function*. σ **always means a standard deviation in this course**; the logistic function is g, never σ
Models	
$\mathcal{F}$	the hypothesis space; $f \in \mathcal{F}$ a model, $\boldsymbol{w}$ its parameters
$\mathcal{F}_0 \subset \mathcal{F}_1 \subset \cdots$	a nested family, indexed by how much freedom is allowed
$\phi_M(\boldsymbol{x})$	the basis functions of order M, grouped into a column
$\boldsymbol{\Phi}$	the design matrix of those basis functions, $n \times (M{+}1)$, one observation per row
$\hat{y}$	the answer the model gives; not always of the same kind as y. For the margin losses (the hinge) it is the score $\eta(\boldsymbol{x})$, not its sign
$\hat{f}$, $\hat{\boldsymbol{w}}$	fitted quantities carry a hat
$\hat{f}_n$	the model fitted to D_n; the subscript recalls that it changes with the sample
η	the linear predictor, or *score*: $\eta(\boldsymbol{x}) = \boldsymbol{w} \cdot \boldsymbol{x} + b$, or $\boldsymbol{w} \cdot \phi_M(\boldsymbol{x})$. In the logistic model it is the log-odds, $\eta = \log \frac{p}{1-p}$; in a linear classifier, $f(\boldsymbol{x}) = \operatorname{sgn} \eta(\boldsymbol{x})$
g	the *activation function*: the fixed, monotone map applied to the linear predictor, $\mu = g(\eta)$. For the logistic model $g(u) = 1/(1+e^{-u})$
h	the *link function* of a generalized linear model, which goes the other way: $h(\mu) = \eta$. The link is what you choose; the activation is what you get, $g = h^{-1}$
Losses and risks	
$\ell(y, \hat{y})$	the loss: the price of answering $\hat{y}$ when the truth is y
$R(f)$	the risk, $\mathbb{E}\big[\ell(y, f(\boldsymbol{x}))\big]$
$R_n(f)$	the empirical risk, $\frac{1}{n}\sum_i \ell(y_i, f(\boldsymbol{x}_i))$ — **averaged, always**
$r(\hat{y} \mid \boldsymbol{x})$	the *conditional risk*: the average loss of answering $\hat{y}$ at that one point, $\mathbb{E}\big[\ell(y, \hat{y}) \mid \boldsymbol{x}\big]$
$\nu(\boldsymbol{x})$	the *noise function*, $\nu(\boldsymbol{x}) = \min_{\hat{y}} r(\hat{y} \mid \boldsymbol{x})$: the best any method can do at that point. Depends on the loss as well as on ρ
$f^\star$	the Bayes predictor, $f^\star(\boldsymbol{x}) = \arg\min_{\hat{y}} r(\hat{y} \mid \boldsymbol{x})$
$R^\star$	the Bayes risk, $R^\star = \mathbb{E}_{\boldsymbol{x}}\big[\nu(\boldsymbol{x})\big]$: the best achievable, averaged over the inputs
$f^\star_{\mathcal{F}}$	the best model *inside* the class, $f^\star_{\mathcal{F}} = \arg\min_{f \in \mathcal{F}} R(f)$. In general $f^\star_{\mathcal{F}} \neq f^\star$, and $R(f^\star_{\mathcal{F}}) - R^\star$ is the approximation error
Regularization and guarantees	
$\Omega(f)$, λ	the regularizer and its weight: $\hat{f}_n = \arg\min_{f \in \mathcal{F}} R_n(f) + \lambda\,\Omega(f)$. λ **is always a penalty weight**

Notation, continued	
τ	the bound of the constrained form of the same problem, $\Omega(f) \leq \tau$
δ	the confidence of a bound: "with probability at least $1-\delta$ over the draw of D_n"
ϑ	a VC dimension
Classification	
ω_k, K	the k-th class; the number of classes
$\Pr(\omega_k)$	the prior of class k
$p(\boldsymbol{x} \mid \omega_k)$	the class-conditional density
$P(\omega_k \mid \boldsymbol{x})$	the posterior
$y \in \{-1, +1\}$	the coding of two classes from the perceptron onwards, so that $y\,\eta(\boldsymbol{x}) > 0$ means "correct"
$\mathbf{1}_{\{\cdot\}}$	the indicator: 1 if the logical condition holds, 0 otherwise
Margins and support vector machines	
γ_i^F	the *functional* margin of observation i, $\gamma_i^F = y_i\,\eta(\boldsymbol{x}_i)$: positive when it is correctly classified
γ_i^G	the *geometrical* margin, $\gamma_i^G = \gamma_i^F / \|\boldsymbol{w}\|$: its signed distance to the hyperplane. The margin of a dataset is the smallest one
B	the radius of the data, $\|\boldsymbol{x}_i\| \leq B$ (as in Novikoff's theorem)
Λ	a bound on the weights, $\|\boldsymbol{w}\| \leq \Lambda$, in the margin bound on ϑ
ξ_i	the *slack* of observation i: how far it is from satisfying $y_i \eta(\boldsymbol{x}_i) \geq 1$
C	the cost of the slacks in the soft-margin SVM, $\frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_i \xi_i$. Dividing by nC gives the course's form, with the hinge loss and $\lambda = 1/(2nC)$
α_i	the dual variables (Lagrange multipliers) of the margin constraints; the *support vectors* are the observations with $\alpha_i > 0$
μ_i	the multipliers of the constraints $\xi_i \geq 0$
Kernels	
$k(\boldsymbol{x}, \boldsymbol{x}')$	a kernel: a similarity that is an inner product in some space, $k(\boldsymbol{x}, \boldsymbol{x}') = \langle \varphi(\boldsymbol{x}), \varphi(\boldsymbol{x}') \rangle_{\mathcal{H}}$
φ	the feature map, $\varphi : \mathcal{X} \to \mathcal{H}$. The basis ϕ_M above is one, of finite dimension
$\mathcal{H}$, $\|f\|_{\mathcal{H}}$	the feature space; for the reproducing kernel Hilbert space (RKHS) of k, its norm, which acts as the regularizer
$\mathbf{K}$	the Gram matrix, $n \times n$, with entries $k_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$
γ	the scale parameter of the Gaussian kernel, $k(\boldsymbol{x}, \boldsymbol{x}') = \exp(-\gamma\|\boldsymbol{x} - \boldsymbol{x}'\|^2)$ — as in the software (see the normalisation section)
μ, $\boldsymbol{v}$	an eigenvalue and an eigenvector in kernel PCA
ε	the width of the tube of the ε-insensitive loss, in support vector regression
$\nu \in (0, 1]$	the parameter of the ν-SVM and of the one-class SVM: a bound on fractions of observations, never a function of $\boldsymbol{x}$
Operators	
$\mathbb{E}$, Var, Cov	expectation, variance, covariance
arg min, arg max	the *argument* attaining the extreme, not its value
$\|\boldsymbol{w}\|_p$	the ℓ_p norm of the parameter vector; $\|\boldsymbol{w}\|$ without subscript is the Euclidean one, $p = 2$
$(u)_+$	the positive part, $\max(0, u)$
sgn	the sign: $+1$ for positive arguments, -1 for negative ones

Conventions. Vectors are columns and are set in bold italic; matrices capitalized in bold upright. Hats mean estimates. The empirical risk is *averaged* over its sample (not summed): this fixes the scale of λ and makes it comparable across sample sizes.

Lower case is pointwise, upper case is averaged. $\ell(y, \hat{y})$ concerns one observation, $r(\hat{y} \mid \boldsymbol{x})$ one input point, and $R(f)$ everything. In particular R **only ever takes a function**: wherever you see

a risk applied to a prediction, it is r.

Symbols with more than one job. The Greek alphabet is short and the literature was not written by one person. A few symbols do two jobs; the form, or the context, tells them apart:

Symbol	Uses	How to tell them apart
ν	noise function $\nu(\boldsymbol{x})$; parameter of the ν-SVM and one-class SVM	the first is a function and always carries its argument; the second is a number in $(0, 1]$
μ	mean of a GLM, $\mu = g(\eta)$; multipliers μ_i of the slack constraints; kernel-PCA eigenvalue	only the multipliers carry an index i; an eigenvalue always comes with its eigenvector
γ	margins γ_i^F, γ_i^G; Gaussian kernel scale	a margin always carries F or G
ε	noise term; tube width of the ε-insensitive loss	the noise is random; the tube width is chosen by you
τ	bound of a constraint $\Omega(f) \leq \tau$; level of the pinball loss	the pinball level lies in $(0, 1)$ and belongs to a loss; the bound belongs to a regularizer
ϕ, φ	basis of order M, ϕ_M; feature map φ	deliberate: the second generalises the first

8 Jargon

The words, without the formulas. Precise definitions come in the lectures; this is so that you are never lost for want of a term.

The setting	
supervised learning	learning a rule from examples that carry the right answer
unsupervised	finding structure with no right answer given
training data	the examples the model is fitted on
validation data	examples held back to *choose* things (the degree, the penalty)
test data	examples held back to *report* a number, used once
feature	an input variable, possibly constructed rather than measured
representation	the whole collection of features the model sees
activation function	the fixed monotone map applied to a linear predictor, written g
link function	the same relation read backwards, from the mean to the linear predictor, written h; so $g = h^{-1}$
target, label	the thing to be predicted; *label* when it is a class
model	one particular rule; *hypothesis* is its synonym here
hypothesis space	the set of rules we allow ourselves to consider
parameter	a quantity the fitting procedure chooses
hyperparameter	a quantity chosen from outside the fitting procedure
Margins and kernels	
margin	how far the observations lie from the decision boundary; the smallest such distance
hard, soft margin	no observation allowed inside the margin; or allowed, at a price C per unit of slack
slack	by how much one observation violates the margin
primal, dual	the problem in the weights; the same problem in one multiplier per observation
support vectors	the observations the solution depends on; all the others could be deleted
kernel	a similarity between two inputs that is an inner product in some feature space
feature map, feature space	the map that sends an input there; the space it lands in
kernel trick	running an algorithm that only uses inner products, with a kernel in their place
Gram matrix	all the pairwise kernel values on a sample, $n \times n$; positive semidefinite
RKHS	the space of functions a kernel defines, with a norm that measures their roughness

Jargon, continued	
representer theorem	the solution of a kernel regularized problem is a combination of the $k(\boldsymbol{x}_i, \cdot)$, one per observation

9 The Rosetta stone

Machine learning	**Statistics**
model	model
parameter, weight	parameter, coefficient
train	fit
learn	infer, estimate
regression	regression
classification	discrimination
clustering	clustering, grouping
inputs, features, variables	independent variables, explanatory variables, predictors, covariates
outputs, targets	dependent variables, response variables
instances, examples	individuals, observations
dataset	sample
error function, loss function	fit criterion, deviance
training error, empirical error	resubstitution error, in-sample error
true error, generalization error	predictive error, out-of-sample error
regularization	shrinkage, penalisation

Beware of neighbouring vocabularies: *transaction* means observation in databases, *attribute* means variable in artificial intelligence.

10 Taxonomies

Orientation only. Only the first is used as an organising idea in class.

- **Generative vs. discriminative** — model the joint $p(x, y)$ and derive the decision, or model $p(y \mid x)$ (or the boundary) directly.
- **Parametric vs. non-parametric** — a fixed number of parameters, or a number that grows with the sample.
- **Linear vs. non-linear** — in the *parameters*, not in the inputs. A polynomial fit is a linear model.
- **Online vs. batch** — one observation at a time, or all at once.
- **Sparse vs. non-sparse** — the solution uses only a few of the observations (the support vectors of an SVM) or of the variables (an ℓ_1 penalty, the lasso); or it uses all of them (ridge regression, kernel ridge regression).
- **Supervised, unsupervised, semi-supervised, reinforcement.**

11 Losses and their optimal predictors

Loss	Definition	Minimiser of the risk
regression: the answer is a number		
squared	$(y-\hat{y})^2$	$\mathbb{E}[y \mid \boldsymbol{x}]$, the conditional mean
absolute	$\lvert y-\hat{y}\rvert$	a conditional median of $y \mid \boldsymbol{x}$
pinball, level τ	$\tau(y-\hat{y})_+ + (1-\tau)(\hat{y}-y)_+$	the conditional τ-quantile
ε-insensitive	$\max\big(0, \lvert y-\hat{y}\rvert - \varepsilon\big)$	any a with $\Pr(y > a+\varepsilon \mid \boldsymbol{x}) = \Pr(y < a-\varepsilon \mid \boldsymbol{x})$
classification: the answer is a label, or a distribution		
0/1	$\mathbf{1}_{\{y\neq\hat{y}\}}$	$\arg\max_k \Pr(y=k \mid \boldsymbol{x})$
logarithmic, or binary cross-entropy	$-\log\hat{p}_y =$ $-y\log\hat{p} - (1-y)\log(1-\hat{p})$	the conditional probability $p(\boldsymbol{x})$ itself
cross-entropy, K classes	$-\sum_{k=1}^{K} \mathbf{1}_{\{y=k\}} \log\hat{p}_k$	the whole posterior vector $\big(P(\omega_k \mid \boldsymbol{x})\big)_k$
hinge, $y \in \{-1,+1\}$, $\hat{y}$ the score	$\max(0, 1-y\hat{y})$	$\operatorname{sgn}\big(2p(\boldsymbol{x})-1\big)$, $p(\boldsymbol{x}) = \Pr(y=+1 \mid \boldsymbol{x})$: the decision of the Bayes classifier

Three remarks worth keeping.

- **The logarithmic loss and the binary cross-entropy are the same thing.** Two communities, two names. The K-class cross-entropy is the genuine generalisation, and it reduces to the binary one when $K=2$ — as the softmax reduces to the logistic.
- **The ε-insensitive loss has a flat bottom**: any prediction within ε of the truth costs exactly nothing. That is not an approximation to the absolute loss, it is a different request — and it is why support vector regression produces solutions that depend on only a few of the observations. At $\varepsilon = 0$ it *is* the absolute loss, and its minimiser is the median.
- **The hinge loss estimates a sign, not a probability**. Its minimiser takes only the values ± 1: it agrees with the Bayes classifier and throws $p(\boldsymbol{x})$ away. This is why an SVM's score $\eta(\boldsymbol{x})$ is not a probability, and why turning it into one needs a separate step (Platt scaling).
- **The minimiser is a functional of $p(y \mid x)$, and the loss chooses which one.** Mean, median, quantile, mode, the whole distribution: changing the loss changes what is being estimated, not merely the arithmetic of estimating it.

12 Exponential family and generalized linear models

Target	Family	Canonical link h	Resulting loss
real	Gaussian	identity	squared
binary	Bernoulli	logit	logarithmic (cross-entropy)
one of K	Multinomial	multinomial logit	cross-entropy (softmax)
count	Poisson	log	Poisson deviance
count of m	Binomial	logit	binomial deviance
positive, skewed	Gamma	inverse (or log)	Gamma deviance
positive with mass at 0	Tweedie	log	Tweedie deviance

In every row the target type selects the family, the family fixes the link h and the loss, and the fitting algorithm follows (IRLS, i.e. Newton–Raphson on the log-likelihood).

Link and activation are inverses of each other. The link maps the mean to the linear predictor, $h(\mu) = \eta$; the activation maps it back, $\mu = h^{-1}(\eta) = g(\eta)$. Most texts write g for the link and g^{-1} for its inverse; *we do not*, because g is fixed here as the activation function, which is the object you will meet again in neural networks. **Caution:** the identity "g is a neural activation function" holds for the logit — where h^{-1} is the logistic — but not for every link in the table above: the inverse probit and the inverse complementary log-log are perfectly good h^{-1} and are not activation functions anyone uses in a network.

13 Normalisation conventions: a warning

The same symbol λ means different things in different software, and a value transplanted from one to another is not the same value.

this course	$\frac{1}{n}\sum_i \ell(y_i, f(x_i)) + \lambda\|w\|_p^p$ (loss averaged, no $1/p$ factor)
`glmnet` (R)	loss averaged, with the customary $\frac{1}{2}$ on the Gaussian term
`Ridge` (scikit-learn)	$\|y - Xw\|^2 + \alpha\|w\|^2$ (loss *summed*, no averaging)
`LogisticRegression`	the *data* term is scaled by C; the penalty is left fixed
SVM: this course	$\frac{1}{n}\sum_i \big(1 - y_i\eta(\boldsymbol{x}_i)\big)_+ + \lambda\|\boldsymbol{w}\|^2$
`svm` (`e1071`), `ksvm` (`kernlab`), `SVC` (scikit-learn)	$\frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_i \xi_i$: loss *summed* and weighted by C. Same solution as ours when $\lambda = 1/(2nC)$; a larger C means *less* regularization

The Gaussian kernel, too. This course, `e1071` and scikit-learn write $\exp(-\gamma\|\boldsymbol{x} - \boldsymbol{x}'\|^2)$, with the argument called `gamma`. `kernlab` writes the same formula with the argument called `sigma`: it is *not* a standard deviation, it is our γ, an inverse squared width. Other texts write $\exp\big(-\|\boldsymbol{x} - \boldsymbol{x}'\|^2/(2s^2)\big)$, where s *is* a width and $\gamma = 1/(2s^2)$.

Conclusion: before using any software routine, check the parameters and results for names, role and default values.

14 Studying with an AI assistant

You may use these tools in this course. What follows is not a set of restrictions dressed up as advice: it is what I would do, and why.

The single distinction that matters is this. **An assistant that gives you the answer has done your work. An assistant that makes you produce the answer has done its own.** Everything below is a way of staying on the second side of that line.

For the exercises: ask for hints, not solutions

An exercise exists so that you get stuck. Getting stuck is not a malfunction of studying; it is studying. An answer handed over intact removes the only part that was doing you any good.

- Ask for **the next step only**: *"I have got as far as this. What is the next move? Do not do it for me."*
- Ask **what kind of object** the answer will be before you look for it: *"Will the answer be a number, a function, or a condition?"*
- Ask it to **check your attempt** rather than replace it: *"Here is my proof. Where is the first step that does not follow?"*

- When you are finished, close the chat and **do the exercise again on paper**. If you cannot, you did not learn it — you watched it.

A good test: could you now explain this to the person next to you, *with the screen off*?

For the theory: make it ask *you*

This is where these tools are genuinely excellent and almost nobody uses them properly. Reading an explanation feels like understanding and is not. Being questioned is uncomfortable and works.

- *"Ask me six questions about the Bayes predictor, one at a time. Do not tell me the answer until I have tried. Then tell me what I got wrong and why."*
- *"I am going to explain the bias–variance decomposition to you. Interrupt me whenever I say something imprecise."*
- *"Give me a statement about regularization that sounds right and is false, and let me find the error."*
- *"What would I have to believe for this result to be false?"*

The last one is worth more than the other three together. It is also the habit this course is trying to build.

These models are confident when they are wrong, and they are wrong most often on exactly the distinctions this subject cares about — two quantities that are nearly the same but not the same. **Treat every explanation as a claim to be checked, not as an authority.**

For the project: the delicate one, because you hand it in

Here the work is assessed, so the line matters legally as well as pedagogically. My advice is to confine the assistance to things that are *craft* rather than *judgement*:

Reasonable	**Not reasonable**
formatting a table; making a figure legible	deciding what the table should show
writing or debugging code you specify	deciding which method to use, or why
finding candidate references	summarising a paper you have not read
translating or tidying your own prose	writing the discussion
explaining an error message	interpreting your results

The right-hand column is what is being marked. If an assistant produces it, there is nothing left for me to assess except its output, and you will have paid for a course you did not take.

- **Code you did not write is code you cannot defend.** You will be asked about your project in an oral examination, and "that is what it gave me" is not an answer.
- **Check every reference it gives you.** These tools invent plausible citations — real authors, real journals, papers that do not exist. **Open every one before you cite it.**
- **Run every line of code it gives you**, on data whose answer you already know.

Citing it: always, and how

Say where you used it, every time — as a comment in the code, and in the text of anything you hand in. The reason is not suspicion. In this course we spend a good deal of time on *why a particular answer is wrong*, and an unattributed generated answer makes that conversation impossible.

The current APA guidance is *Citing generative AI in APA Style* (September 2025), which replaced the earlier advice on citing ChatGPT:

https://apastyle.apa.org/blog/cite-generative-ai-references

In short: the AI is never an author — the company is. A reference has the form

Company. (year, month day). *Title of the chat* [Generative AI chat]. Tool/Model. URL

and where a shareable link to the conversation exists, include it. For anything that is not a quotation — code you adapted, a figure it helped you format, prose it tidied — a sentence in the text is enough, saying which tool, which version, and what it did.

For this course a line like the following, in the report and beside the code, is exactly what I want:

```
# The plotting code below was drafted with <tool, version>, <date>,
# from my specification, and then checked and modified by me.
```